Comparison of Implementing a PID Controller into an Arduino with Theoretical Data

4/29/2026

Joe Parker, Robert Danner and Luke Clemens

Dr. Valente

Disclaimer GradeScope only allowed PDF uploads, and the video could not be uploaded as a secondary file. The video presentation can be accessed through YouTube using this link:

https://www.youtube.com/watch?v=fo38JWNDqhM

Introduction

Feedback controller design is necessary to accurately track a motor at a desired speed while rejecting real-world disturbances that an open loop input alone cannot compensate for. By designing a PID controller informed by the system's identified transfer function, the motor can be driven to meet specified performance requirements such as settling time and overshoot.

This report analyzes the open and closed-loop behavior of the DC motor system identified in the previous project and implements a controller on the Arduino to compare theoretical simulation results against experimental measurements. Based on the derivation from the previous project, the identified transfer function is given in Equation 1.

$$G(s) = \frac{K}{\tau s+1} \quad (1)$$

This theoretical model of the transfer function was seen to be a second order system, but as this report will explain, the system can be written as a first order system, as seen in equation 1. This is because first order systems have no oscillatory behavior therefore negating the need for the derivative gain which dampens such behavior.

Open-Loop Analysis

In project 1, the DC system gain represented by K was found to be 31.2375, and the time constant of the system, represented by τ, was found to be 0.159s. This indicates the motor will reach 63.2% of the steady state speed within 160ms of a step input. This brings the final open loop transfer function to be

$$G(s) = \frac{31.2375}{0.159s + 1} \quad (2)$$

The open loop response is shown in figure 1.

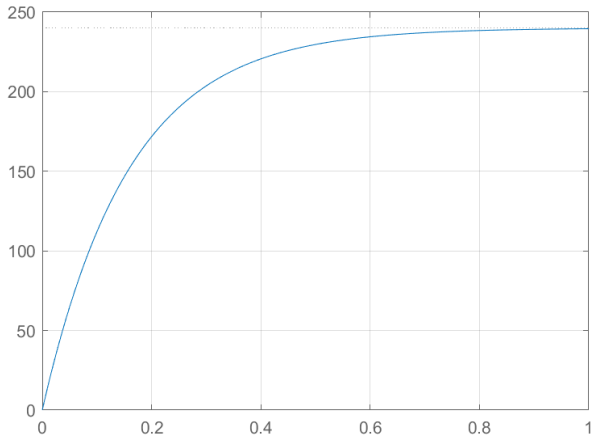


Figure 1. Open-Loop RPM Response over time in seconds

The open-loop response exhibits an exponential rise, which is characteristic of our first-order transfer function.

Poles and Zeros

The open loop transfer function in equation (2) has no finite zeros since no value of S can make the numerator equal to zero. Analyzing the same transfer function, one pole can be found when solving the denominator equal to zero and solving for S. A single pole can be found for this first order equation at S = -6.29 rad/s. Since the only pole lies on the negative real axis, the open loop system is inherently stable and will have no tendency to oscillate.

Step Response

The open loop step response for the system was set with a step input reference speed of 80% of the nominal motor speed. The maximum speed of the motor was found to be 300 RPM; thus, the reference speed was set to be 240 RPM. The open loop output can be found by multiplying the transfer function by the step magnitude in the Laplace Domain. This is shown in equation 3:

$$Y(s) = G(s) \cdot (240/s) = \frac{31.2375}{0.159s + 1} \cdot (240/s) \quad (3)$$

The open loop response exhibits the characteristic behavior of a first-order system. Seen in figure 1, the response rises smoothly and has zero overshoot; this is consistent with a system having one real pole and no zeros. The rise time is defined as the time it takes for the system to rise from 10% to 90% of the steady state value. The rise time for a first order system is approximately 2.2 times the time constant. Thus bringing the rise time of the open loop system as 0.349 seconds. This is consistent with the value of 0.349 seconds obtained from MATLAB's stepinfo command.

Closed-Loop Analysis

Figure 2 shows the closed-loop response of the system with the y-axis marking the motor RPM's and the x-axis denoting the time in seconds after sending power to the motor.

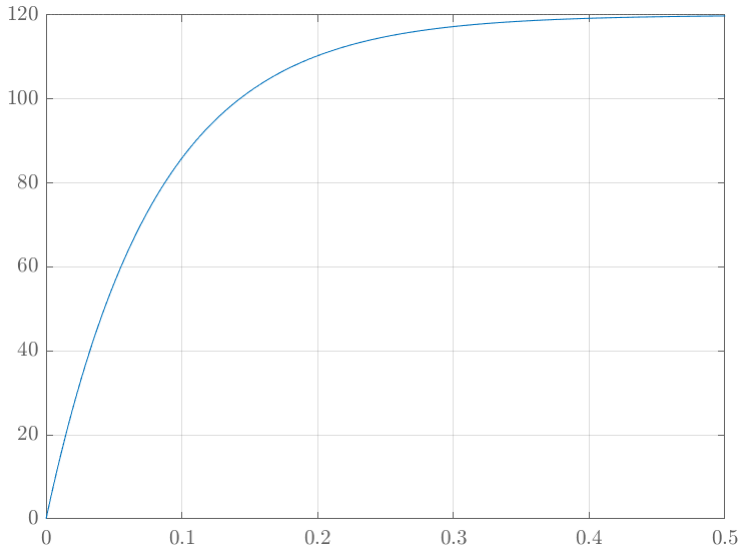


Figure 2. Closed-Loop RPM Response over time in seconds

Unity Feedback Closed-Loop Transfer Function

With unity feedback, the closed loop transfer function is:

$$T(s) = \frac{G(s)}{1 + G(s)} = \frac{31.2375}{0.159s + 32.2375} \tag{4}$$

This closed loop transfer function yields a new pole when solving the denominator equal to zero to be at s=-202.7 rad/s. The new closed loop time constant can be found by dividing 0.159/32.2375. This results with a time constant $\tau_{cl} = 0.00493\ s$ or 4.93ms. The closed loop DC gain can be found by dividing 31.2375/32.2375 yielding a gain of 0.9693. Clearly adding the closed feedback loop dramatically accelerates the transient response when compared to the open-loop response. This can be seen by comparing the time constant of open-loop and closed-loop. The open loop response reaches 63% of its steady state value at 160ms, while the closed-loop response reaches 63% of its steady state value at 5ms, clearly showing a much faster response.

Step Response and Time-Domain Response

Putting the transfer function into its standard first-order form:

$$T(s) = \frac{0.9693}{0.00493s + 1} \tag{5}$$

Using a step input of magnitude 240 RPM, the Laplace domain response is

$$Y(s) = T(s) \cdot (240/s) = \frac{0.9693}{0.00493s + 1} \cdot (240/s) \tag{6}$$

Taking the inverse Laplace transform gives the time domain response as

$$y(t) = 232.6 \cdot \left(1 - e^{\frac{-t}{0.00493}}\right) \tag{7}$$

This equation is used when time is greater than zero and shows how the motor speed evolves over time with unity feedback of k=1. Here 232.6 RPM is the steady state value the system approaches at time goes to infinity. It is equal to the 240RPM times the DC gain of 0.9693.

Rise Time, Settling Time, and Final Value

The rise time as previously mentioned is the time it takes the system to go from 10% of its steady state value to 90% of its steady state value. For the closed loop system, the rise time is approximately 2.2 times the closed-loop time constant t_r = 2.2 * 0.00493 = 0.0108 seconds or about 10.8ms. The settling time, with 2% margin, is approximately 5 * τ_cl = 24.6 ms. The final value is 232.6 RPM, which reflects the 3.1% steady state error due to the unity feedback DC gain of 0.9693.

Root Locus Analysis

Root locus is a tool used to examine how the closed-loop poles move in the complex s-plane as the feedback gain (K) varies between 0 and infinity. The closed loop characteristic equation is defined as:

$$1 + K \cdot G(s) = 0 \tag{8}$$

which for this system gives:

$$1 + K \cdot \frac{31.2375}{0.159s + 1} = 0 \tag{9}$$

Rearranging gives the final closed loop characteristic equation:

$$0.159s + 1 + 31.2375K = 0 \tag{10}$$

The Root Locus of the system was plotted in MATLAB and is shown in figure 3.

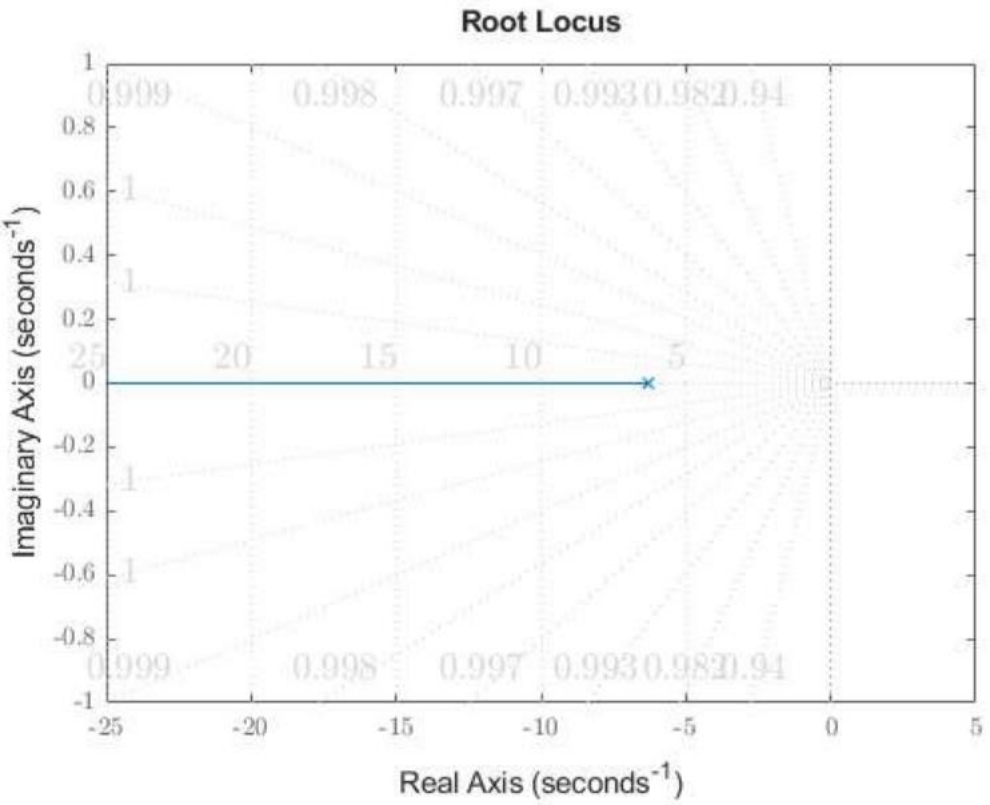


Figure 3. Root Lucus Plot in Complex Plane

The root locus plot in figure 3 confirms that the closed loop system has a single pole that originates at s=-6.29 rad/s when K=0. The pole then shifts leftward towards minus infinity as K rises. The root locus shows that the pole stays entirely on the negative real axis which confirms the stability of the system. Since the locus in entirely real, all closed loop poles have a damping ratio of $\zeta = 1$.

Proportional Controller Evaluation

The root locus analysis allows evaluation of whether a proportional controller alone can satisfy the design requirements of a settling time of less than 5 seconds and an overshoot less than 10%. Since the root locus lies entirely on the negative real axis for all K>0, the closed loop pole is always real. This means the damping ratio is always equal to 1, and the overshoot is 0% for any positive gain. This satisfies the overshoot requirement. For the settling time requirement, the closed loop pole at K=1 is located at s=-202.7 rad/s, which gives a setting time of approximately 4/202.7 = 0.020 seconds or 20ms. This is well within the 5 second limit given. As K further increases, the poles move farther to the left and the and the settling time further improves. Therefore, a proportional controller does meet both the settling time and overshoot requirements for K>0. However, since the plant is a type 0 (meaning no poles at the origin) the proportional

controller can't eliminate the steady state error. The error decreases with increasing K but never reaches zero. This limitation calls for a PI controller with integration to achieve zero steady state error.

PID Controller Design

The goal for this PID Controller Design was to refine the closed-loop step response. To optimize the response, the main objective was to eliminate the steady-state error that was introduced by the unity feedback earlier, while minimizing settling time and overshoot. Using MATLAB, Kp and Ki values were adjusted to meet these criteria. The following transfer function was found:

$$\mathbf{C(s)} = 2 + \frac{15}{s} = \frac{2s+15}{s} \tag{11}$$

When multiplied with the plant, the following transfer function was found with our previous transfer function:

$$\mathbf{G(s)} = \frac{31.24}{0.159s + 1} \tag{12}$$

$$\mathbf{C(s)G(s)} = \left(\frac{31.24}{0.159s + 1}\right)\left(\frac{2s+15}{s}\right) = \left(\frac{62.48s + 468.8}{0.159s^2+s}\right) \tag{13}$$

Therefore, the final closed-loop transfer function with the plant and the controller is:

$$\left(\frac{62.48s + 468.8}{0.159s^2+63.48s+468.8}\right) \tag{14}$$

The controller transfer function is based around the form: C(s) = Kp + Ki/s + Kds. Kd was designed to be 0 because there was no oscillatory behavior that needed damping from our open and closed loop responses. Kp, which is the proportional term, was set to 2 to improve the speed of the response. The integral term, Ki, was set to 15 to effectively eliminate the steady-state error that was observed in the closed-loop response.

	Open-Loop	**Closed-Loop**	**PID**
Rise Time (s)	0.3493	0.1747	0.1695
Settling Time (s)	0.622	0.311	0.2909
Overshoot (%)	0	0	0.07
Peak Value (RPM)	239.84	119.92	240.17

As the results show, the rise and settling times have slightly improved from the closed-loop times. The proportional gain of 2 was kept low to avoid introducing significant oscillation and overshoot. The steady-state error introduced in the closed-loop section has been fixed, and the system now settles at 240 RPM with less than 0.1% overshoot.

Experimental Implementation and Results

Arduino Implementation

The implementation to Arduino was to design a controller in the Arduino IDE with the same circuit setup as the previous project, seen in figure. The gain variables obtained in the previous sections were put into the code for controlling the current error (Kp), the error over time (Ki) and the speed at which the error changes (Kd). The sample time is the same for every 100ms, the code calculates the RPM scaling it to the PWM to print the RPM output at each interval.

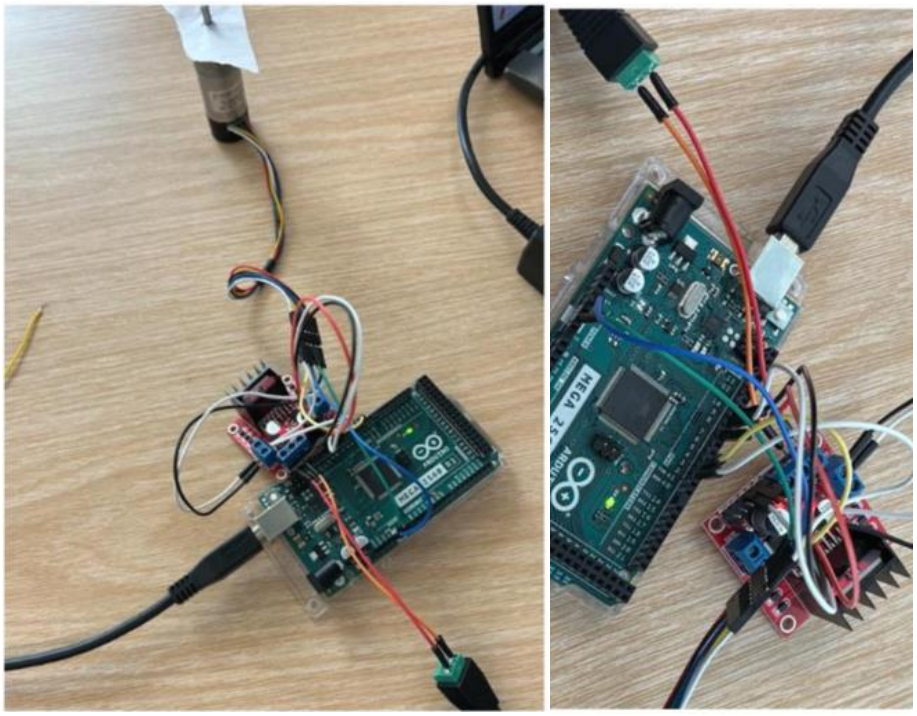


Figure 4. The circuit setup for the Arduino with a zoomed out (left) and a zoomed in (right) versions.

After running the Arduino code and getting a steady state output near 240 RPM, the data was saved as an Excel file. A MATLAB code was used to process the data and compare it to computed results using built-in MATLAB functions.

Comparison of Experimental and Simulated Results

The Arduino implementation successfully drove the motor to the target speed of 240 RPM, representing 80% of the motor's 300 RPM rated maximum. Figure shows strong agreement between the MATLAB-

simulated response and the experimentally measured Arduino response, confirming the feasibility of the identified transfer function as a model of the physical system.

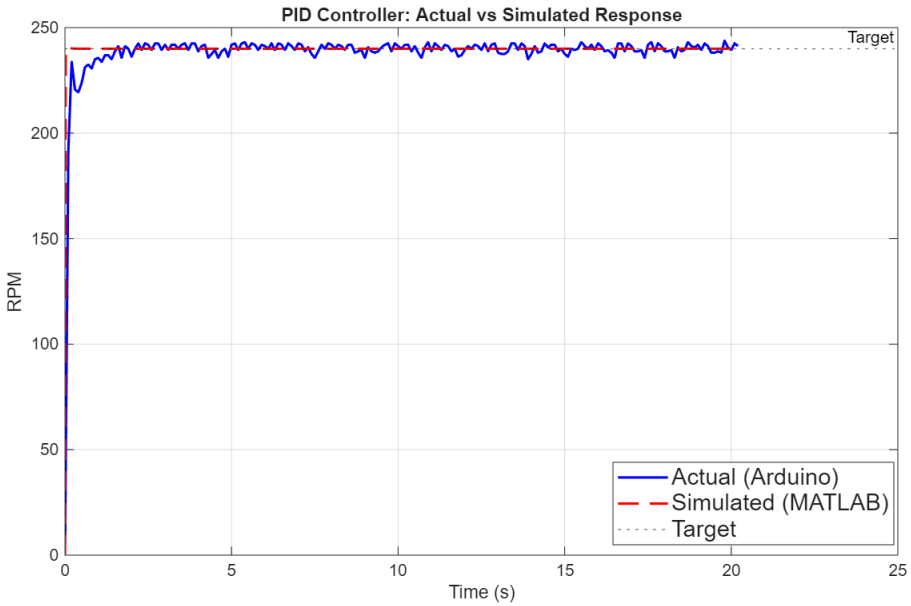


Figure 5. Response curves of the simulated and Arduino code runs compared to the intended target RPM.

The actual Arduino response exhibits a jagged pattern throughout steady state, which is attributable to noise inherent to the encoder-based RPM measurement rather than true speed variation in the motor. In contrast, the MATLAB simulation assumes ideal continuous-time conditions with no measurement noise, producing a smooth flat line once steady state is reached. Together these two behaviors account for the visual discrepancy between the curves despite the close agreement in steady-state value.

The controller was implemented as a PI controller, with the derivative gain set to zero. This is appropriate given that the plant was identified as first-order, a purely mechanical system with negligible electrical dynamics, which exhibits no oscillatory behavior that derivative action would need to damp. The persistent high-frequency noise visible in the measured response is an artifact of the encoder-based RPM calculation rather than true speed variation and does not affect the steady-state performance of the controller.

Conclusion

This report analyzed the open-loop and closed-loop behavior of a DC motor system and implemented a PID controller on an Arduino to optimize the system. The first-order transfer function, which had a DC gain of $K = 31.24$ and a time constant $\tau = 0.159$ seconds, served as the basis for all simulations and controller design.

The unity feedback closed-loop response revealed a significant steady-state error, which enhanced the need for a PI controller with $Kp = 2$ and $Ki = 15$. The resulting controller successfully eliminated the steady-state error while achieving a settling time of 0.2909 seconds and a near-zero overshoot of 0.07%, satisfying the design requirements. The derivative term was found to be unnecessary since the plant was a first-order transfer function, which shows no oscillatory behavior.

The experimental Arduino implementation confirmed strong agreement between the simulated and measured responses, as the motor successfully reached the target speed of 240 RPM. The only discrepancy between the two was attributed to noise. Overall, the identified transfer function proved to be an accurate model of the physical system, and the PI controller demonstrated its effective performance.